



PA 3 Report

2024010779 刘家豪

实现方法

参考实现的主要问题有两点：

1. **warp divergence 严重**：如果让一个线程负责一整行，行度数差异会导致同一 warp 内线程工作量很不均匀；
2. **访存局部性较差**：SpMM 的主要开销来自对稠密矩阵 `B` 的不规则读取，若不同 warp 处理的行访问模式差异过大，缓存命中率较低。

因此我的实现思路包括：

- 按 `K=32` 和 `K=256` 分别设计 kernel
- 在 `preprocess()` 中按行度对 row 做分桶
- 对不同桶使用不同的线程映射
- 对部分 row list 做轻量排序，改善对 `B` 的访问局部性
- 对极高度数行单独处理，缓解 load imbalance

K=32 的实现

`K=32` 的实现将行分为 low rows、light rows、hub rows 三类。

- 对于度数很小的行（默认 `deg <= 8`，部分图上放宽到 `deg <= 16`），使用 `spmm_kernel32_low4`，此 kernel 中
 - 一个 warp 同时处理 4 行；
 - 每 8 个 lane 负责一行；
 - 每个 lane 处理一个 `float4`，从而覆盖全部 32 个 feature。
- 对于普通行，核心思路是不再让一个线程串行处理整行，而是让整个 warp 协同完成一行：`spmm_kernel32_rows` 进行 4-way edge split，`spmm_kernel32_rows8` 进行 8-way edge split，`spmm_kernel32_rows16` 进行 16-way edge split。
 - warp 内不同子组处理同一行的不同 edge 子区间；
 - 每个线程仍使用 `float4` 处理连续特征；
 - 最后通过 `__shfl_down_sync` 做 warp 内归约。

- 对于极高度数行，普通的 one-warp-per-row 方式会导致该 warp 执行时间过长，因此单独使用 `spmm_kernel32_hub`。

- 一个 hub row 切成多个子区间；
- 每个子区间由一个 block 处理；
- 结果通过 `atomicAdd` 累加到输出行。

由于 hub kernel 会对同一输出行做多次累加，因此在每次 `run()` 前使用

`spmm_clear_rows32` 只清零对应 hub rows，避免旧结果残留，同时避免对整个输出矩阵做额外清零。

K=32 中不同数据集的度分布差异很大，因此我没有使用固定阈值，而是基于 `avg_deg` 与 `max_deg` 做启发式选择，在不对单个数据集硬编码的前提下，尽量同时兼顾长尾图（如 `arxiv`）和平均度较高的图（如 `amazon_cogdl`、`reddit.dgl`）。

- 自适应 `hub_threshold`；
- 自适应 `low_threshold`；
- 自适应选择 `split4 / split8 / split16`；
- 对 `light rows` 是否排序也使用启发式控制。

K=256 的实现

K=256 时，特征维度较大，单行计算量也更高，因此实现方式与 K=32 有一定的差异。最终实现分为 light rows、heavy rows、hub rows 三类。

- 对普通行使用 `spmm_kernel256_rows`，保持实现简单，同时获得较好的寄存器复用。
 - 一个 warp 处理一行；
 - 每个 lane 负责 8 个特征位置（间隔 32）；
 - 在寄存器中累计 8 个部分和，再一次性写回。
- 对于度数较高的行，使用 `spmm_kernel256_heavy` 缓解高度数行上的负载不均衡：
 - 一个 block（16 warps）协同处理一行；
 - 不同 warp 负责不同 edge 子区间；
 - warp 的部分结果写入 shared memory；
 - 最终由 warp 0 汇总并写回输出。
- 对于极高度数行，进一步使用 `spmm_kernel256_hub`：
 - 一个 hub row 切成 32 个子区间；
 - 每个子区间由一个 block 处理；
 - 最终通过 `atomicAdd` 汇总到输出。

同样地，在每次运行前仅清零 hub rows 对应输出，避免原子累加污染结果。

在 `K=256` 中，

- 将 `heavy_threshold` 调低到 16，使更多高度数行进入协同处理；
- 对 heavy rows 和 hub rows 按 `first_idx = idx[ptr[row]]` 排序；
- 通过 `gather_first_idx_kernel` 在 GPU 上批量提取排序键，避免逐行 `cudaMemcpy`。

这样可以在不引入复杂图重排的前提下，提高相邻 warp 对稠密矩阵 `B` 的访问局部性。

优化路径

K=32

`K=32` 的最终优化是逐步叠加得到的，平均吞吐量（`avg(nnz/t)`）变化如下：

阶段	主要改动	平均吞吐量
baseline	one warp per row 的初始实现	约 2.88e9 nnz/s
step 1	light rows 改为 split4	约 3.46e9 nnz/s
step 2	增加 low4 packing	约 3.75e9 nnz/s
step 3	去掉 <code>K=32</code> heavy rows，普通行统一走 split light kernel	约 4.58e9 nnz/s
step 4	light row sorting, adaptive hub threshold	约 5.21e9 nnz/s
step 5	增加 split16 与 low-threshold 策略	5.47e9 nnz/s

最有效的优化主要来自三点：

1. 改变线程映射，减少 warp divergence；
2. low-degree rows packing，提高 warp 利用率；

3. 按 first idx 排序，提高访存局部性。

以 `products` 和 `amazon_cogdl` 这类大图为例，`light rows` 按 `first_idx` 排序后，相邻 warp 更容易访问到稠密矩阵 `B` 的相近行，因此缓存命中率更高，性能提升比较明显。在实验中，这一优化对以下数据集提升较大：

- `reddit.dgl` : 22.8ms -> 16.4ms
- `products` : 33.1ms -> 24.7ms
- `amazon_cogdl` : 58.7ms -> 49.1ms
- `ppa` : 10.6ms -> 8.2ms

以 `arxiv` 为例，其平均度不高但存在明显长尾，因此更适合较低的 `hub_threshold`，让极高度数行更早进入 hub kernel，从而缓解 load imbalance。

K=256

`K=256` 的优化重点包括

- 将高度数行切换为多 warp 协同处理；
- 对极高度数行使用更高的拆分粒度；
- 通过排序改善对 `B` 的局部性。

最终 `K=256` 的平均吞吐量达到 `6.83e8 nnz/s`。

实验结果

K = 32

- 平均吞吐量: `5.47e9 nnz/s`
- 超过 `cuSparse` 的数据点数: **13 / 13**

dataset	opt(us)	cuSparse(us)	speedup
arxiv	351.15	731.66	2.08x
collab	551.93	1270.42	2.30x

dataset	opt(us)	cuSparse(us)	speedup
citation	8987.56	16445.9	1.83x
ddi	249.72	640.43	2.56x
protein	7371.20	24652.0	3.34x
ppa	8236.78	18356.5	2.23x
reddit.dgl	16024.3	48561.6	3.03x
products	24705.7	55827.3	2.26x
youtube	1592.51	3642.12	2.29x
amazon_cogdl	45119.5	125317.0	2.78x
yelp	3400.41	6575.01	1.93x
wikikg2	2478.16	7140.33	2.88x
am	1690.04	3742.11	2.21x

K = 256

- 平均吞吐量: **6.83e8 nnz/s**
- 超过 cuSparse 的数据点数: **11 / 13**

dataset	opt(us)	cuSparse(us)	speedup
arxiv	2582.79	2992.64	1.16x
collab	4381.54	5202.40	1.19x
citation	65902.9	78877.6	1.20x
ddi	1680.04	1553.18	0.92x
protein	103280.0	80834.9	0.78x
ppa	60925.0	84886.6	1.39x
reddit.dgl	154200.0	202326.0	1.31x

dataset	opt(us)	cuSparse(us)	speedup
products	177415.0	258348.0	1.46x
youtube	11990.9	14405.9	1.20x
amazon_cogdl	425491.0	517023.0	1.22x
yelp	28135.5	29985.5	1.07x
wikikg2	14765.6	16646.4	1.13x
am	10493.9	13398.5	1.28x